

0.69.4. La fonction `sprite()` — un lutin au pixel près

`#include <sprite>` : `sprite(numero, x, y, tuile)` place un lutin — une tuile qui bouge pixel par pixel, par-dessus le fond.

1. Ce qu'il faut comprendre

Ce qu'elle fait : un **lutin** est une tuile qui n'est pas sur la grille : il se place au **pixel près**, par-dessus le fond.

`sprite(numero, x, y, tuile)` place le lutin `numero` (0 à 39) en (x, y).

La ligne à écrire : `#include <sprite>`.

Ses arguments : le numéro du lutin, x de 0 à 159, y de 0 à 143, la tuile ; un cinquième, facultatif, le retourne (`MIROIR_X`, `MIROIR_Y`).

Ce qu'elle coûte : environ **145 octets** la première fois : les 40 lutins rangés, et leur copie à chaque image.

Essaie : fais-le descendre : `y = y + 1;`.

On retrouve `sprite` dans 49 leçons

[0.70](#), [0.70.1](#), [0.70.2](#), [0.108](#), [43](#), [45](#), [46](#), [48](#), [49](#), [50](#), [51](#), [83](#), [85](#), [99](#), [100](#), [104](#), [106](#), [107](#), [108](#), [109](#), [110](#), [110.3](#), [110.4](#), [110.5](#), [110.6](#), [110.7](#), [110.8](#), [110.9](#), [110.11](#), [110.13](#), [111](#), [112](#), [113](#), [122](#), [124](#), [125](#), [127](#), [128](#), [131](#), [132](#), [134](#), [136](#), [137](#), [140](#), [143](#), [152](#), [2.35](#), [2.37](#), [2.39](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **224 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	50 o	← la console, à l'allumage — la mise en route de la console
main()	39 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
RangerLutins	11 o	← démarrage — range les 40 lutins hors de l'écran
InstallerTransfert	15 o	← démarrage — installe la copie des lutins en mémoire rapide
CopierTuiles	31 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
Tuiles	16 o	← CopierTuiles — 1 lettre sur 43
DonneesTransfert	10 o	← InstallerTransfert — la petite routine qui copie les lutins

Lettres gravées : A

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreAcces`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `EffacerCarte`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `Notes`

Fonctions du langage appelées :

<code>sprite()</code>	× 1	23 o sur place, dans <code>main()</code>
<code>image()</code>	× 1	6 o sur place, dans <code>main()</code> + <code>AttendreImage</code> (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- #include <sprite> : un lutin au pixel près ----  
// Le A avance d'un pixel par image, de gauche à droite.  
  
#include <sprite>      // sprite() : un lutin, au pixel près  
#include <ALPHABET>
```

2. UNE VARIABLE — X

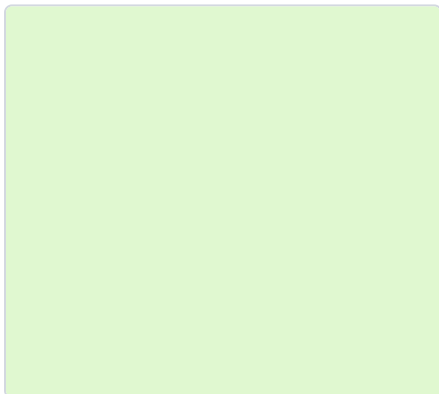
```
uint8_t x = 0;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

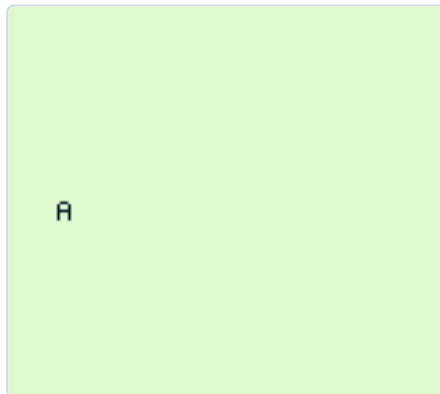
```
int main() {  
    while (true) {  
        image();  
        x = x + 1;           // un pixel de plus à chaque image  
        sprite(0, x, 72, ALPHABET[0]); // le lutin 0, en (x, 72)  
    }  
}
```

3. Ce qui se passe, étape par étape

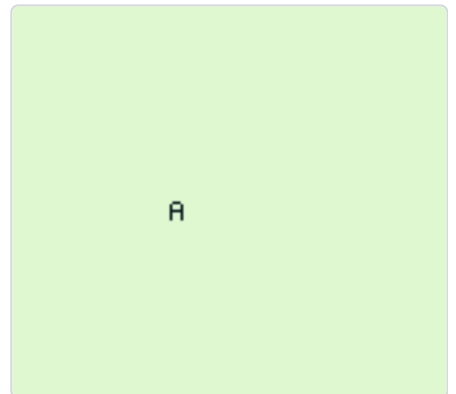
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



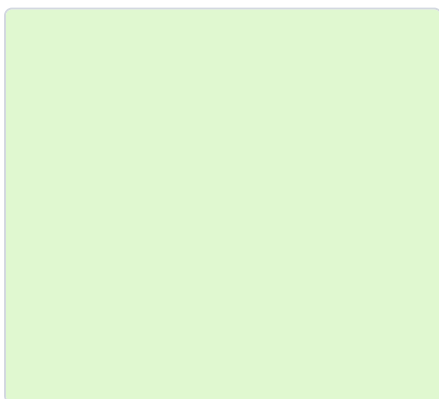
la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180

4. Ce qu'on doit voir

Un A qui glisse doucement de gauche à droite, au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le lutin 0 porte le A
- ✓ il est à la hauteur 72
- ✓ il avance d'un pixel par image (+10)

Ensuite : **0.70. Plus fluide : la lettre au pixel près** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++